

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 2

Important Note: Please download Isabelle from the link provided in the slides and bring a laptop to the large group tutorial. Performing the proofs in Isabelle will be an integral part of those tutorials.

Before you start, please open a new Isabelle/HOL theory file called `Week_1.thy` and add the following to its beginning.

```
theory Week_2
imports Main
begin
```

Exercise 2.1 Addition is Associative

Consider the function $(+)$ defined in Isabelle/HOL for natural numbers.

Prove the following theorem, first pen-and-paper, and then formally in Isabelle.

lemma *add_associative*: $((a::nat) + b) + c = a + (b + c)$

- Hints:**
1. Perform the proof by induction, using substitutions *subst*.
 2. Think about which variable to perform induction on. Addition on natural numbers is defined by recursion on its first argument: look at *thm add_0_left add_Suc*.
 3. You will need to identify multiple lemmas about how *add* works. Do not prove those lemmas. Use *sorry* as the proof. This is the top-down approach.
 4. There is a proof which uses only the following lemmas: *add_0_left*, *add_Suc*

Exercise 2.2 Truncated Subtraction

Subtraction on natural numbers is truncated: $n - m = 0$ whenever $n \leq m$. Prove that, as long as $m \leq n$, subtracting m and adding it back gives n again. You should prove it both, pen-and-paper and in Isabelle, by induction on n .

lemma *sub_add*: $(m::nat) \leq n \implies (n - m) + m = n$

- Hints:**
1. In the step case, perform a case analysis with *proof(cases m ≤ n)*. In the second case, together with the assumption $m \leq \text{Suc } n$, you can derive $m = \text{Suc } n$.

2. Use *thm Suc.IH* in the step case to look at the induction hypothesis. It has an assumption, which you can discharge with *OF*.
3. The following lemmas suffice: *le0*, *le_antisym*, *diff_self_eq_0*, *add_0_left*, *Suc_diff_le*, *add_Suc*, *not_le_imp_less*, *Suc_leI*.

Exercise 2.3 Induction Hypothesis Generalisation with an Accumulator

Consider the following function, which adds two natural numbers using an accumulator: in each recursive call it moves one *Suc* from its first argument to its second.

```
fun add_acc :: nat ⇒ nat ⇒ nat where
  add_acc 0 m = m
| add_acc (Suc n) m = add_acc n (Suc m)
```

We want to prove that *add_acc* computes addition.

lemma *add_acc_correct*: *add_acc n m = n + m*

(a) First try to prove the lemma by plain induction on *n*, i.e. with *proof(induction n)*. Write down the goal of the step case and the induction hypothesis. Explain, pen-and-paper, why the induction hypothesis is not strong enough to prove the step case.

(b) Now prove the lemma, first pen-and-paper and then formally in Isabelle, by generalising the induction hypothesis over *m*, i.e. with *proof(induction n arbitrary: m)*. Your pen-and-paper proof should state the generalised induction hypothesis and say with which term you instantiate its universally quantified variable in the step case.

Hints: 1. *add_acc* is a recursive function defined with *fun*, like *add* in Section 2.2.2 of Concrete Semantics. Its defining equations are available as *add_acc.simps(1)* and *add_acc.simps(2)* (look at *thm add_acc.simps*), and can be used with *subst* like any other equation.

2. Use *thm Suc.IH* in the step case to compare the induction hypothesis you get with and without *arbitrary: m*.

3. The following lemmas suffice: *add_acc.simps*, *add_0_left*, *add_Suc*, *add_Suc_right*.

(c) Challenge (optional) The following function multiplies using an accumulator. Prove, pen-and-paper and in Isabelle, that it computes *acc + n * m*. Which variable must you generalise over this time, and why?

```
fun mult_acc :: nat ⇒ nat ⇒ nat ⇒ nat where
  mult_acc 0 m acc = acc
| mult_acc (Suc n) m acc = mult_acc n m (acc + m)
```

lemma *mult_acc_correct*: *mult_acc n m acc = acc + n * m*

Hint: The following lemmas suffice: *mult_acc.simps*, *mult_0*, *add_0_right*, *mult_Suc*, *add.assoc*.